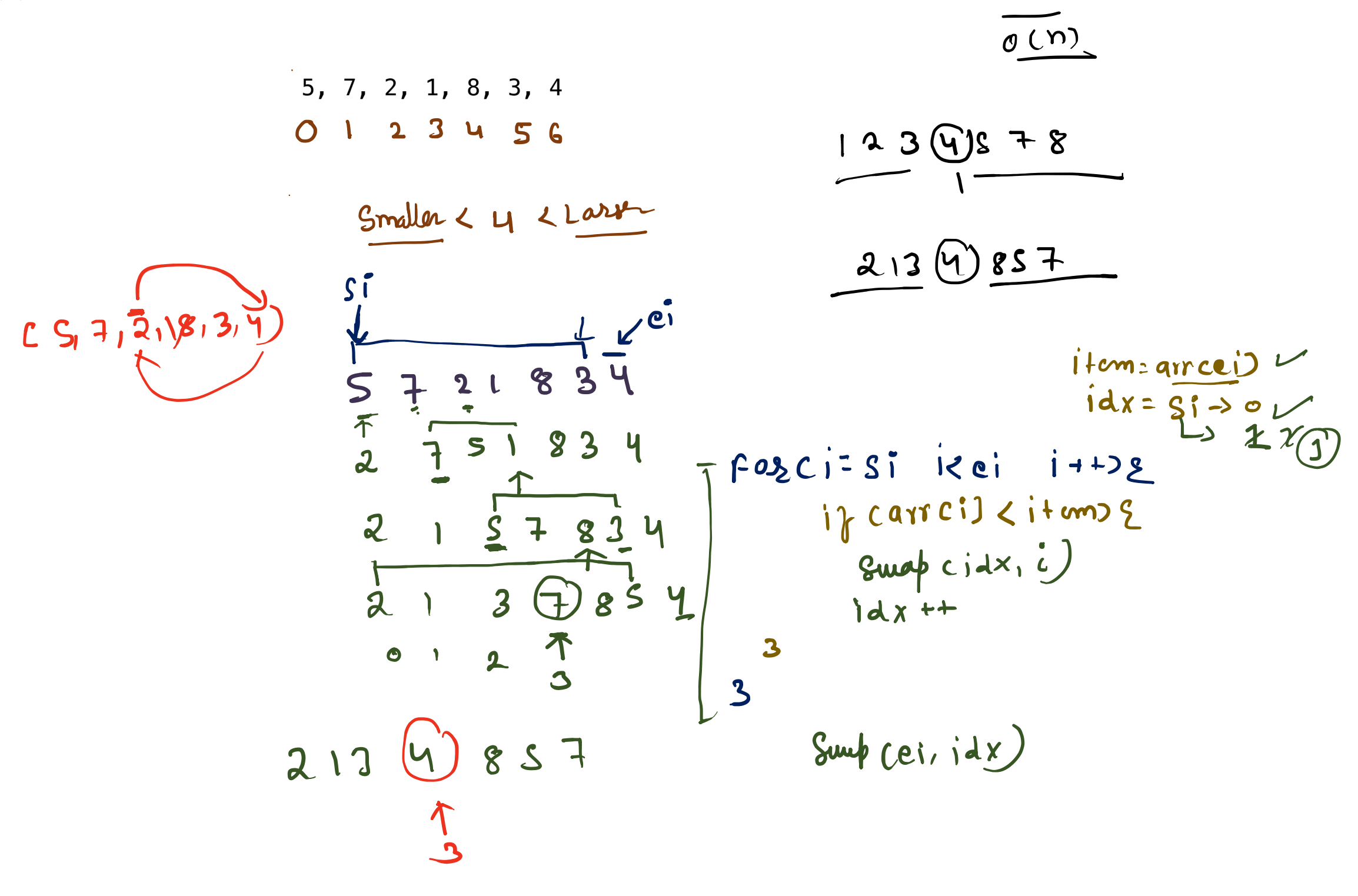
Lec-14

30/06/26 10:19 AM


5, 7, 2, 1, 8, 3, 4
0 1 2 3 4 5 6
O(n)
Smaller < 4 < Larger
1 2 3 4 5 7 8
2 1 3 4 8 5 7
[5,7,2,1,8,3,4]
si
ei
5 7 2 1 8 3 4
2 7 5 1 8 3 4
2 1 5 7 8 3 4
2 1 3 7 8 5 4
0 1 2 3
item = arr[ei]
idx = si → 0 1 2 3
for(i = si; i<ei; i++){
if(arr[i] < item){
swap(idx, i)
idx++
swap(ei, idx)
2 1 3 4 8 5 7

Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* `s`.

Example 1:

```
Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"
Output: [["a"]]
```

aab

[a,a,b] [aa,b]

nitin

n|i|t|i|n

n|iti|n

nitin

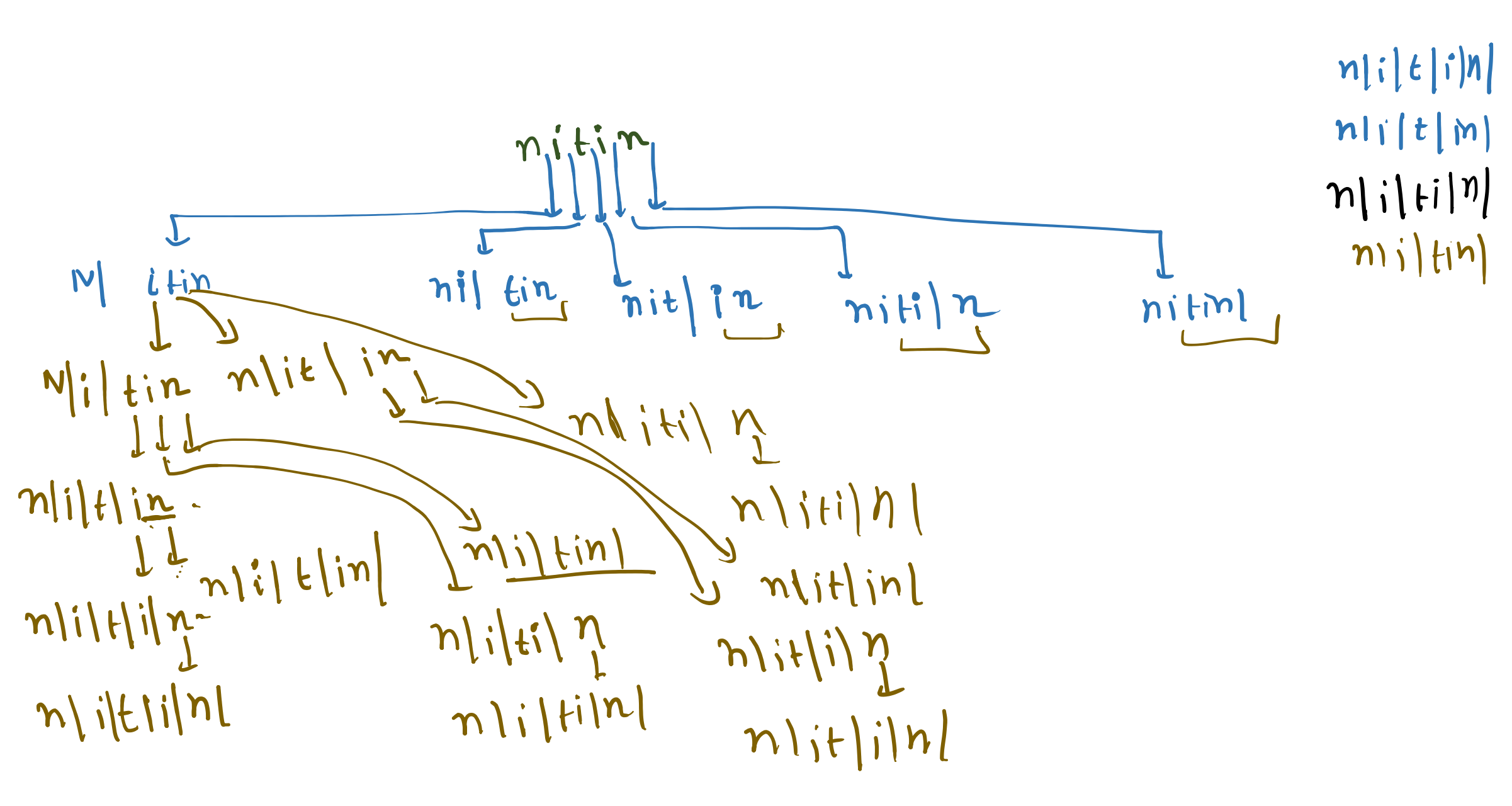
abc →

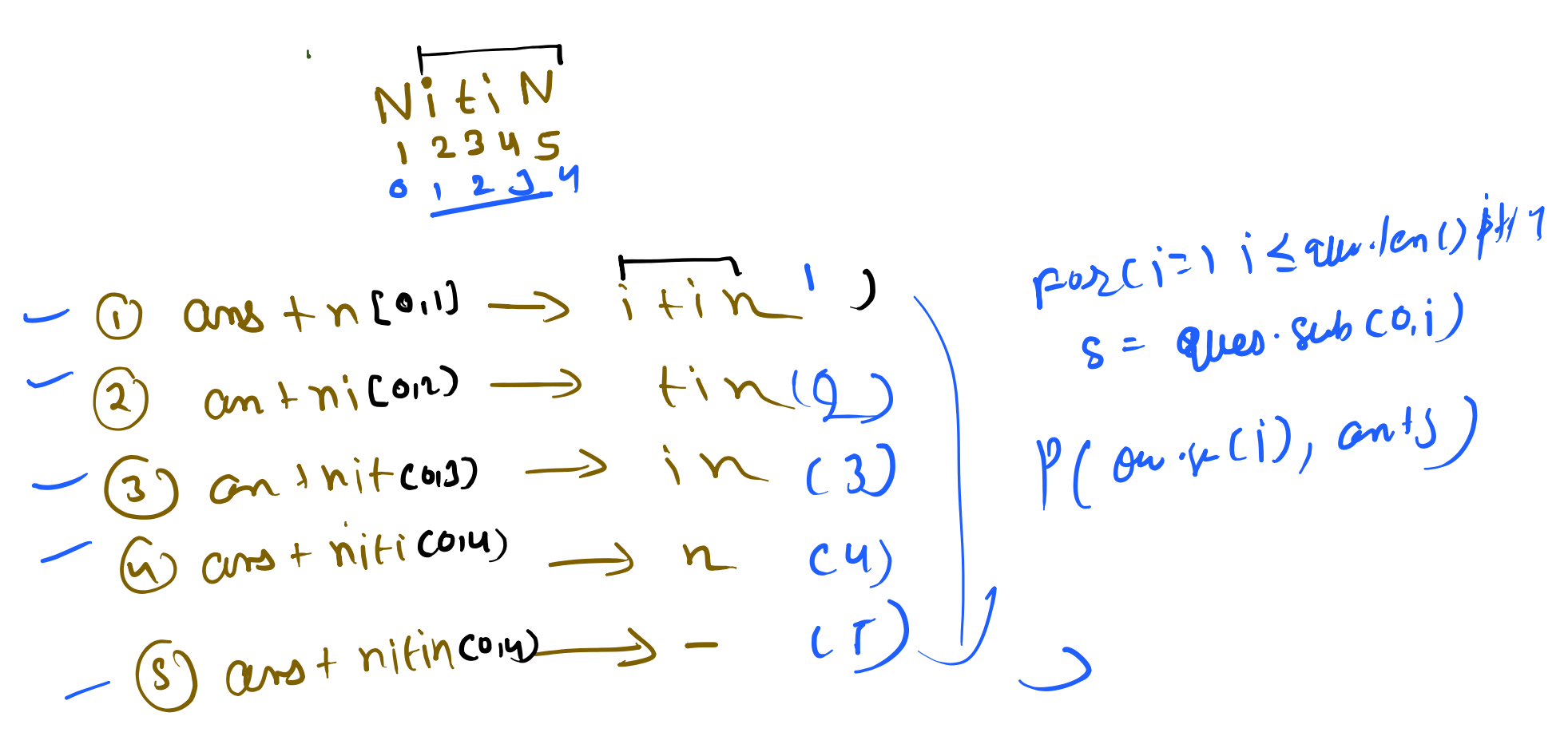
a|b|c

ab|c

a|bc

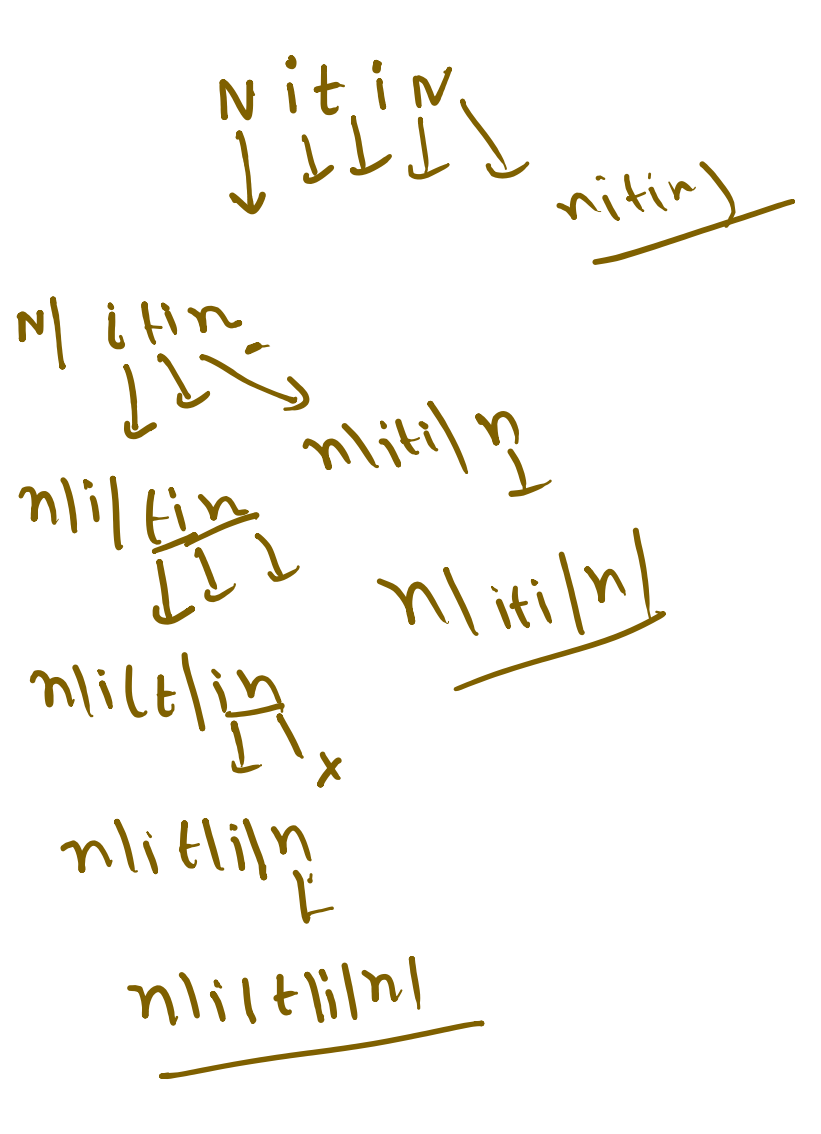
abc

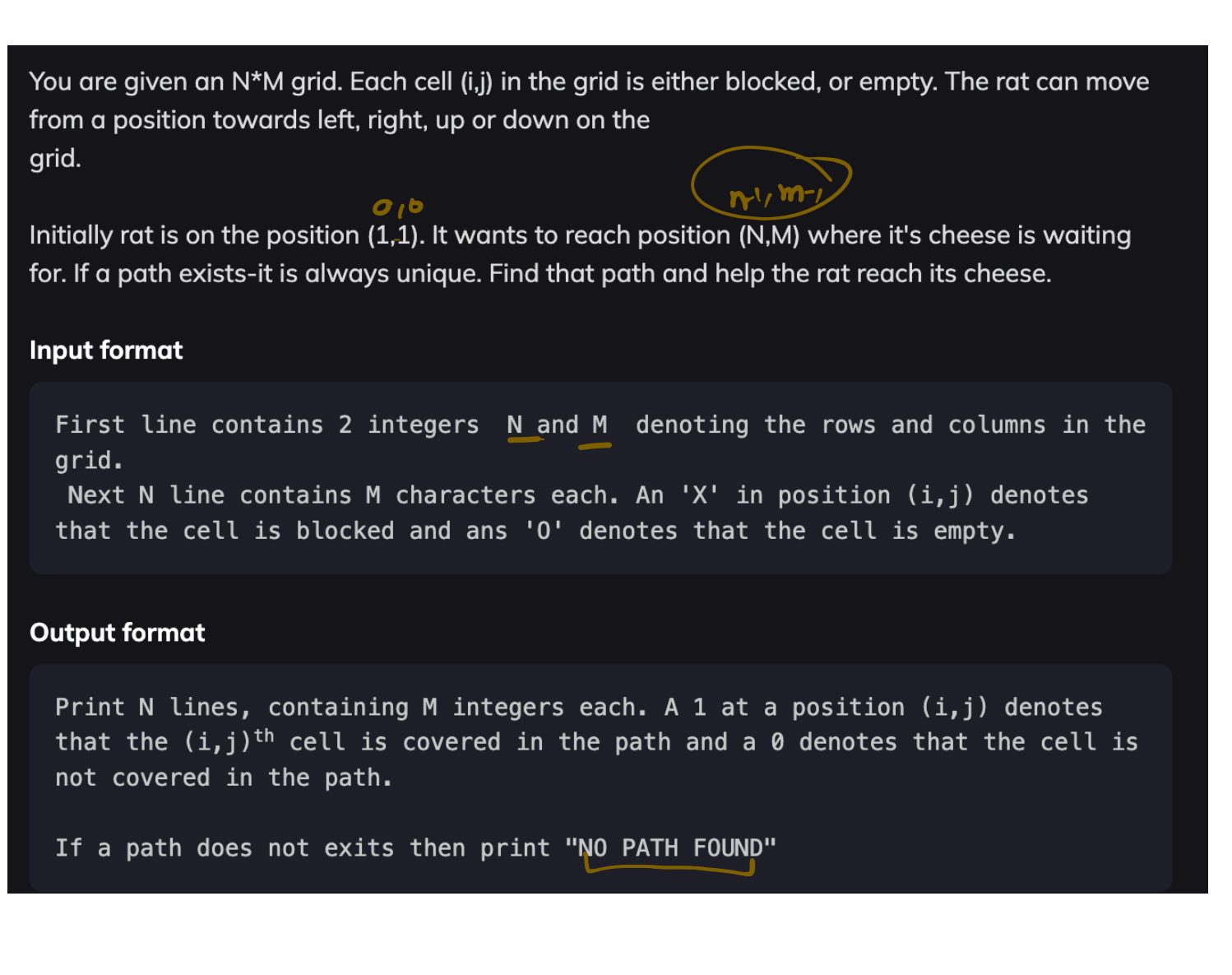

nitin
N| itin
ni| tin
nit| in
niti| n
nitin|
N|i|tin
N|it|in
N|iti|n
N|i|t|in
N|i|ti|n
N|i|tin|
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|it|i|n|
n|iti|n|
n|i|t|i|n|
n|i|t|in|
n|i|t|i|n|
n|i|tin|


NitiN
1 2 3 4 5
0 1 2 3 4
① ans + n [0,1] → itin (1)
② an + ni [0,2] → tin (2)
③ an + nit [0,3] → in (3)
④ ans + niti [0,4] → n (4)
⑤ ans + nitin [0,4] → – (5)
for(i=1 i≤ques.len() i++)
s = ques.sub(0,i)
P(ques.s(i), ans+s)

```
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|
n|itin|
ni|t|i|n|
ni|t|in|
ni|ti|n|
ni|tin|
nit|i|n|
nit|in|
niti|n|
nitin|
```

```java
public static void Partition(String ques, String ans) {
        if (ques.length() == 0) {
            System.out.println(ans);
            return;
        }
        for (int i = 1; i <= ques.length(); i++) {
            String s = ques.substring(0, i);
            if (isPalindrom(s)) {
                Partition(ques.substring(i), ans + s + "|");
            }

        }
    }

    public static boolean isPalindrom(String s) {
        int i = 0;
        int j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
```


Nitin
nitin|
N| itin
n|i|tin
n|iti|n
n|i|t|in
n|iti|n|
n|i|t|i|n
n|i|t|i|n|



You are given an N*M grid. Each cell (i,j) in the grid is either blocked, or empty. The rat can move from a position towards left, right, up or down on the grid.

Initially rat is on the position (1,1). It wants to reach position (N,M) where it's cheese is waiting for. If a path exists-it is always unique. Find that path and help the rat reach its cheese.

Input format

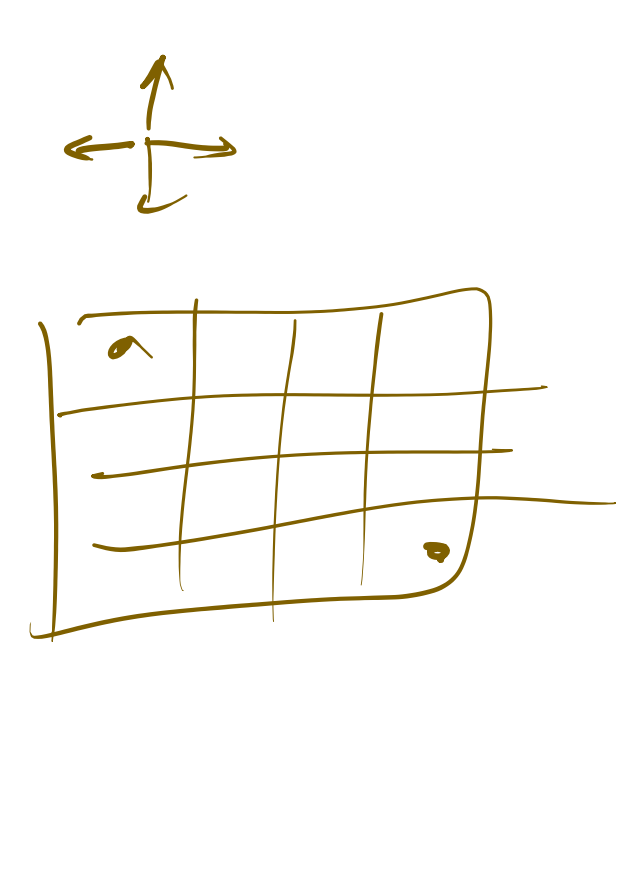
```
First line contains 2 integers  N and M  denoting the rows and columns in the
grid.
 Next N line contains M characters each. An 'X' in position (i,j) denotes
that the cell is blocked and ans 'O' denotes that the cell is empty.
```

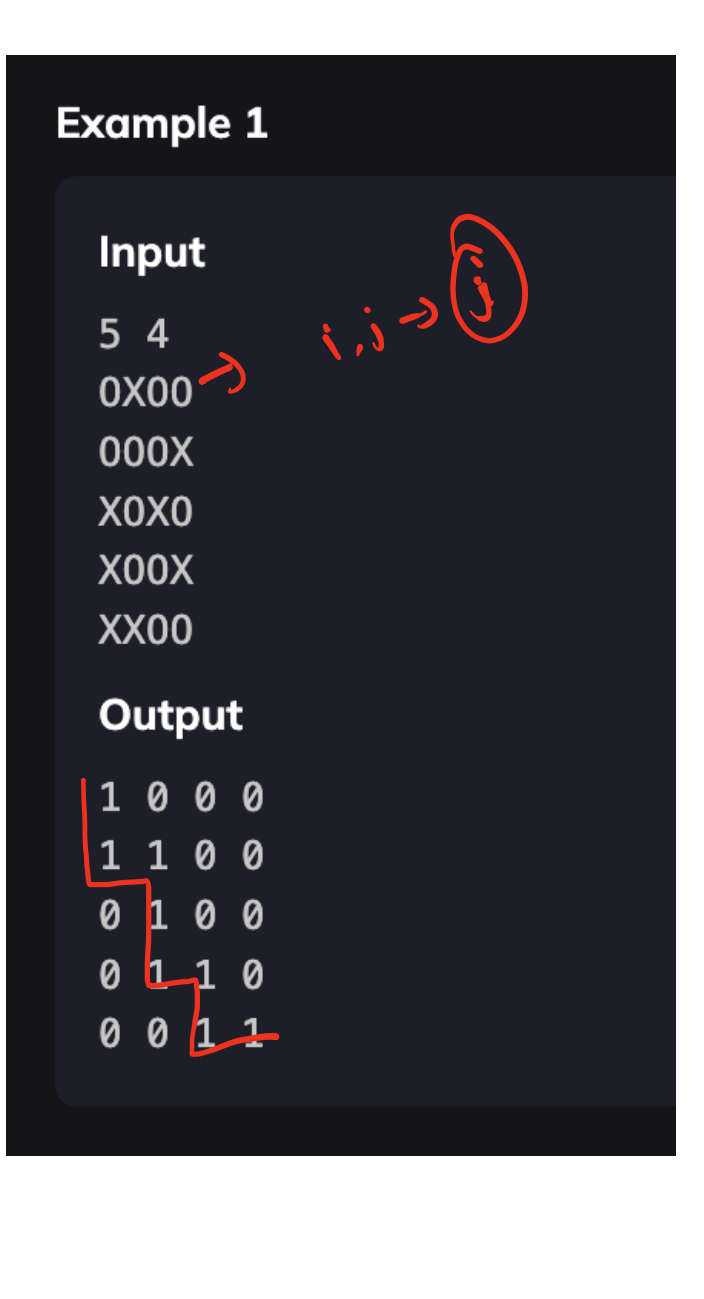
Output format

```
Print N lines, containing M integers each. A 1 at a position (i,j) denotes
that the (i,j)th cell is covered in the path and a 0 denotes that the cell is
not covered in the path.

If a path does not exits then print "NO PATH FOUND"
```

0,0

n-1, m-1





Example 1

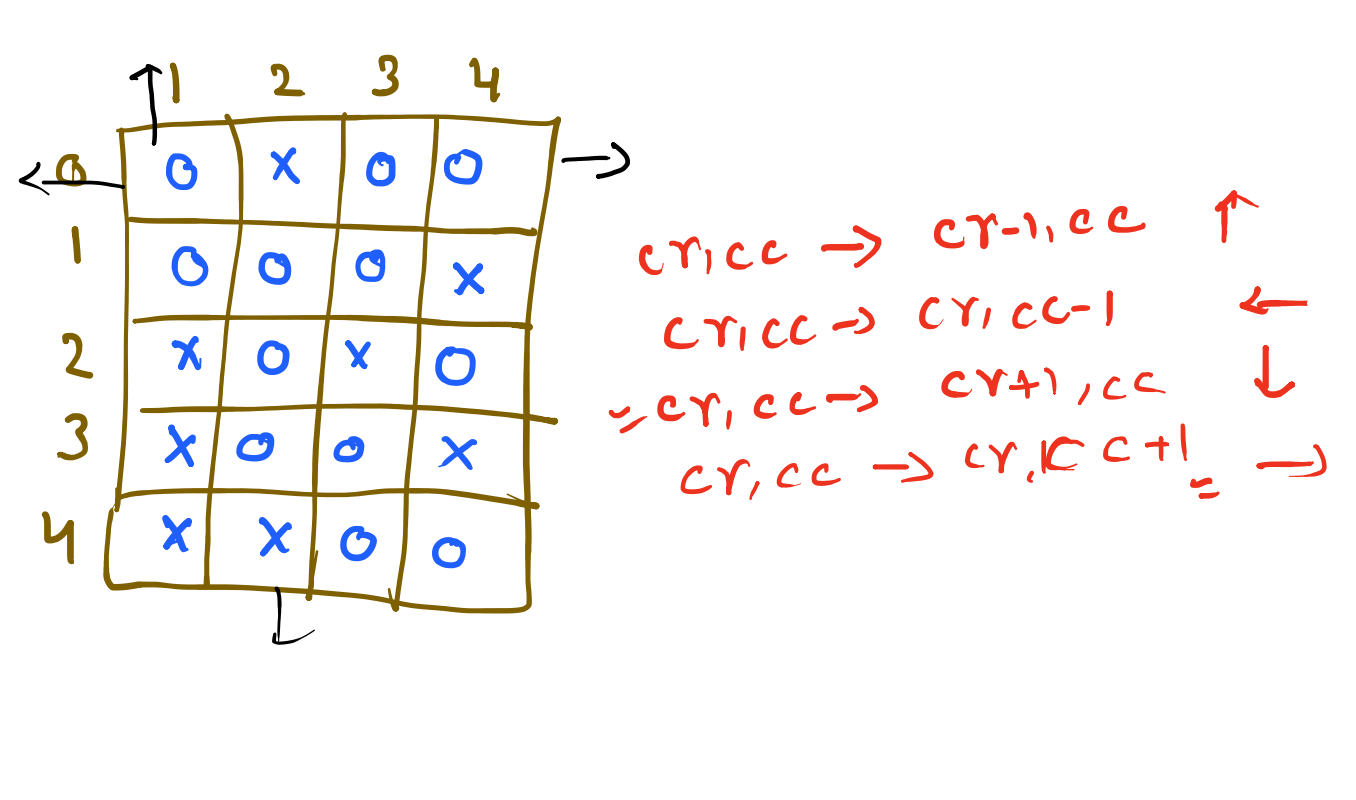
Input

```
5 4
0X00
000X
X0X0
X00X
XX00
```

Output

```
1 0 0 0
1 1 0 0
0 1 0 0
0 1 1 0
0 0 1 1
```

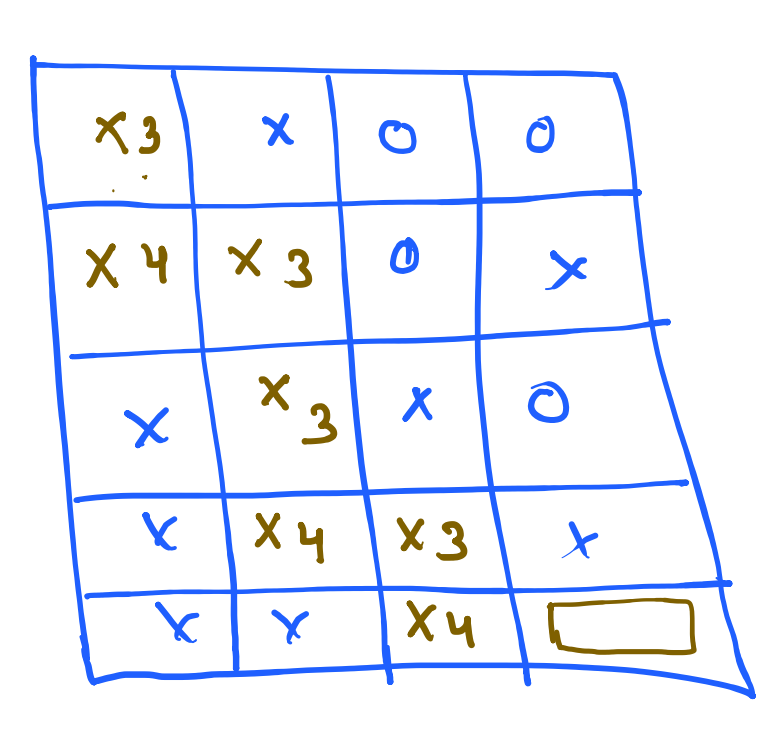
i,j → (j)


1 2 3 4
0 1 2 3 4
O X O O
O O O X
X O X O
X O O X
X X O O
cr,cc → cr-1,cc ↑
cr,cc → cr,cc-1 ←
cr,cc → cr+1,cc ↓
cr,cc → cr,cc+1 →

```java
public static void print(char[][] maze, int cr, int cc) {
        // TODO Auto-generated method stub
        if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length||maze[cr][cc]=='X') {
            return;
        }
        maze[cr][cc]='X';
        print(maze, cr - 1, cc);// up
        print(maze, cr, cc - 1);// left
        print(maze, cr + 1, cc);// down
        print(maze, cr, cc + 1);// right
        maze[cr][cc]='O';

    }
```

– ans(cr)(cc) = 1

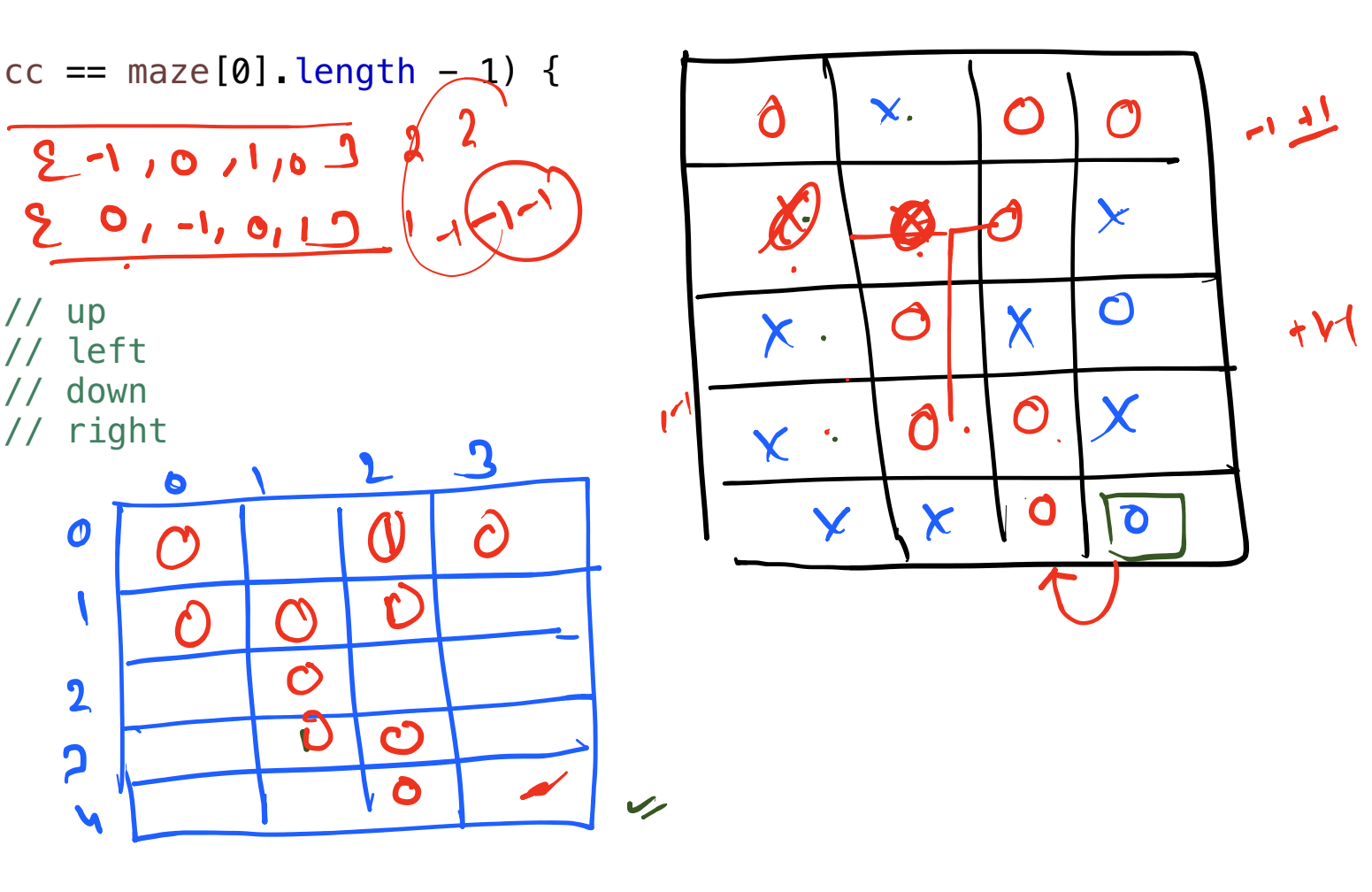
ans (cr)(cc) = 0

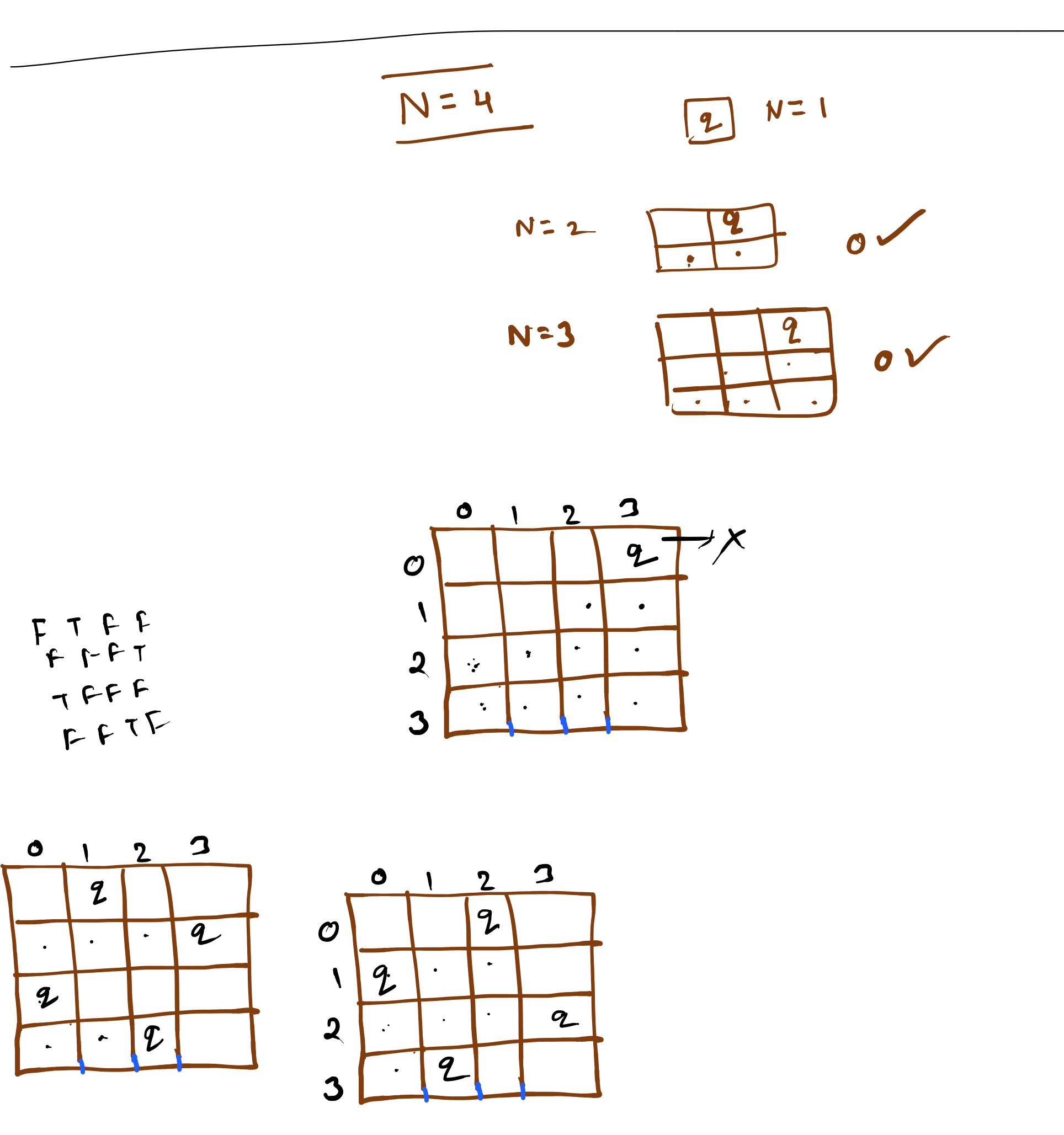

X3 X O O
X4 X3 O X
X X3 X O
X X4 X3 X
X X X4

```java
public static void print(char[][] maze, int cr, int cc, int[][] ans) {
        // TODO Auto-generated method stub
        if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length || maze[cr][cc] == 'X') {
            return;
        }
        if (cr == maze.length - 1 && cc == maze[0].length - 1) {
            Display(ans);
            return;
        }
        maze[cr][cc] = 'X';
        ans[cr][cc] = 1;
        print(maze, cr - 1, cc, ans);// up
        print(maze, cr, cc - 1, ans);// left
        print(maze, cr + 1, cc, ans);// down
        print(maze, cr, cc + 1, ans);// right
        maze[cr][cc] = 'O';
        ans[cr][cc] = 0;

    }
```

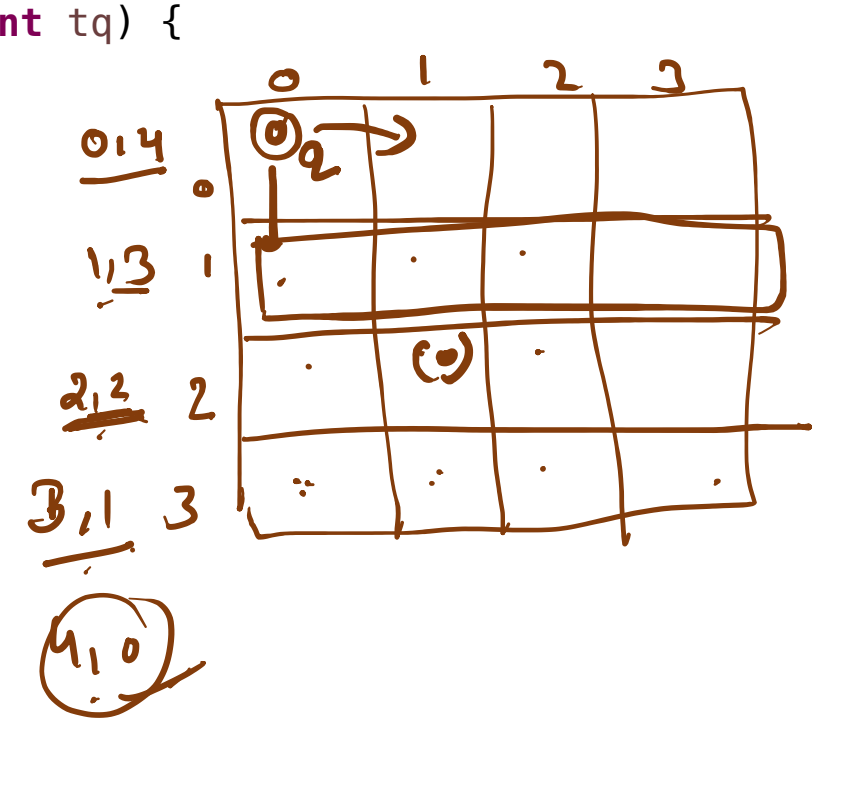
R = {-1, 0, 1, 0}

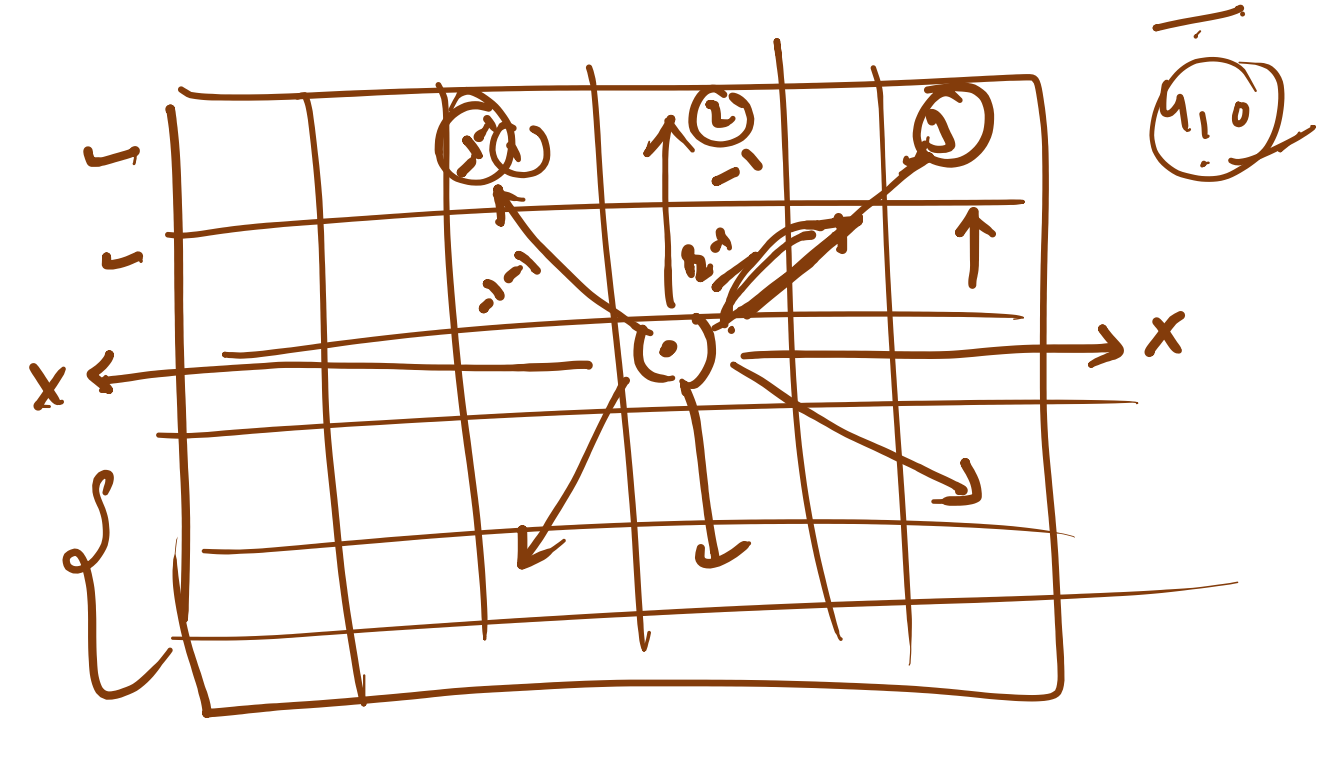
C = {0, -1, 0, 1}

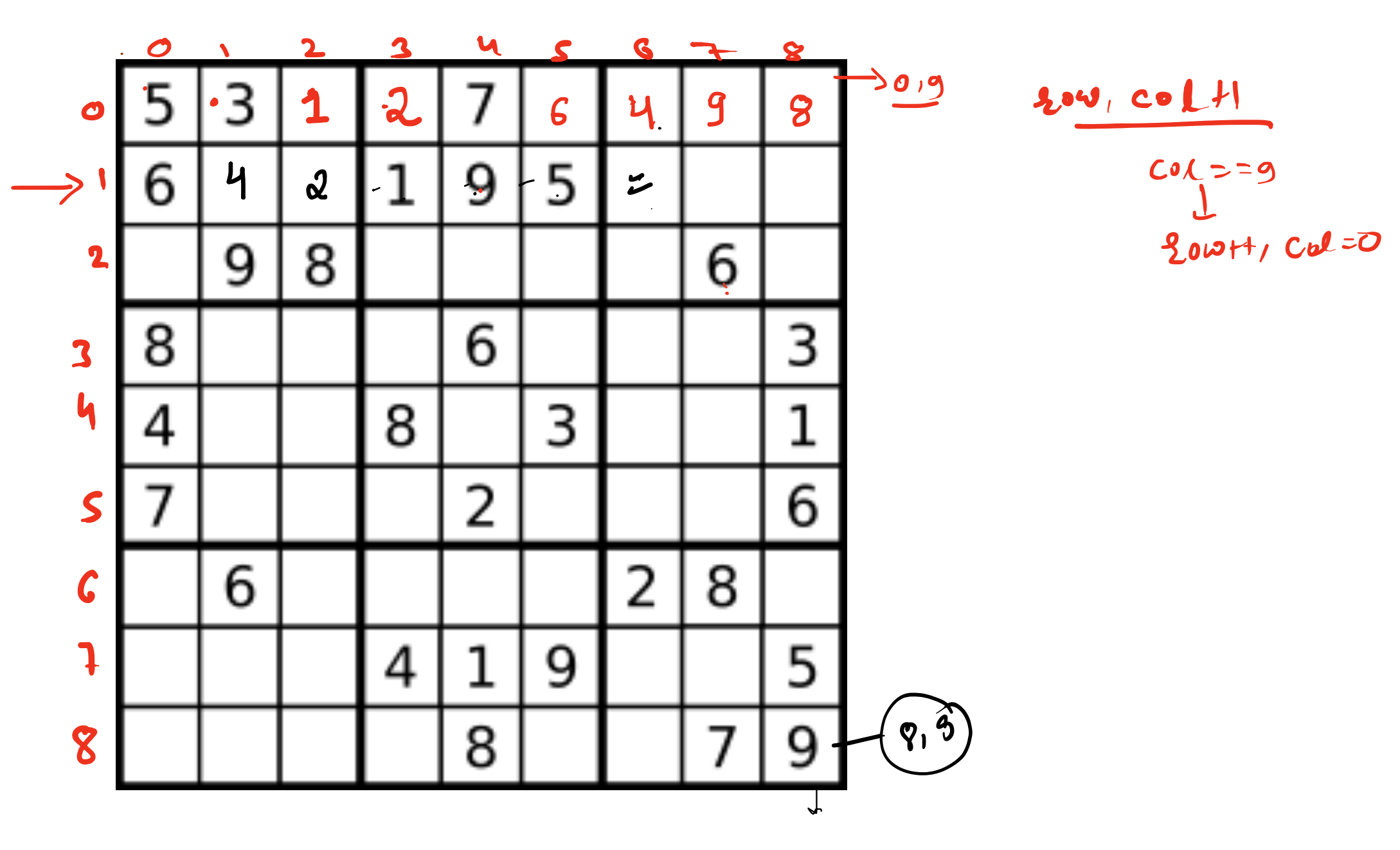

2
-1 -1
+1
-1
0 1 2 3
0 1 2 3 4


N=4
q N=1
N=2 0 ✓
N=3 0 ✓
0 1 2 3
q
X
F T F F
F F F T
T F F F
F F T F

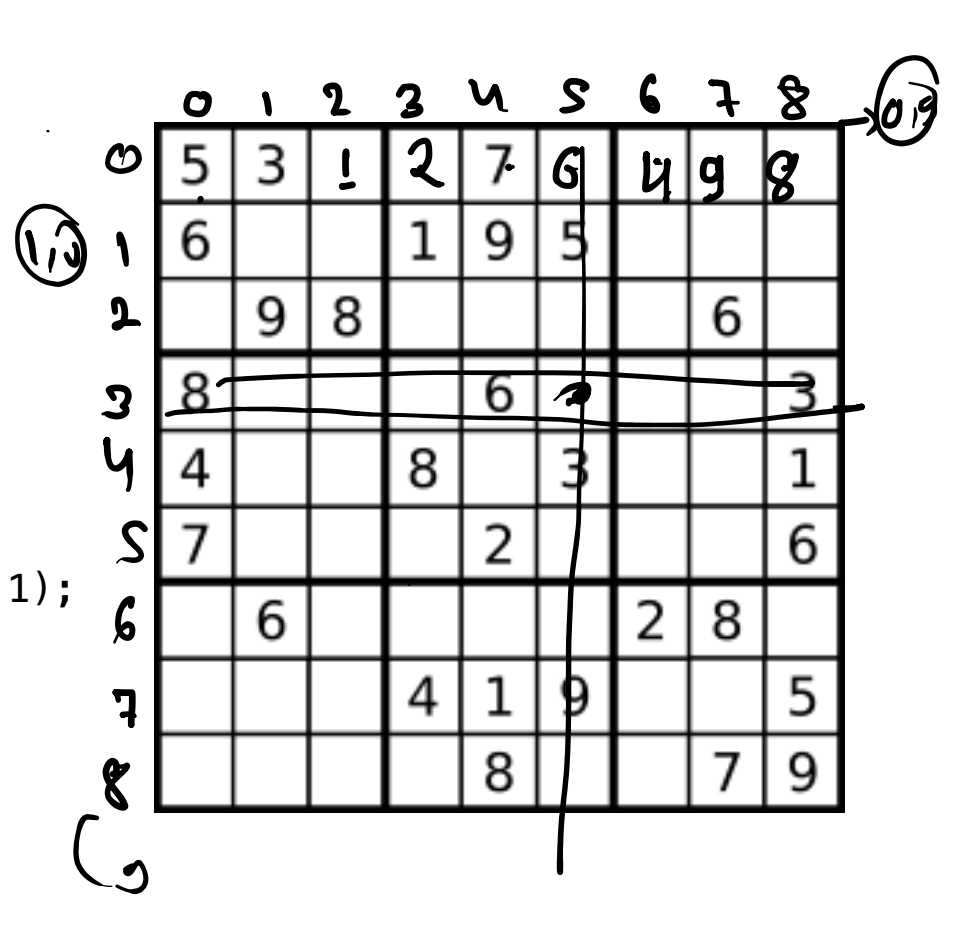
```java
public static void print(boolean[][] board, int row, int tq) {
        if (tq == 0) {
            Display(board);
            return;
        }
        for (int col = 0; col < board.length; col++) {
            if (isitsafe(board, row, col)) {
                board[row][col] = true;
                print(board, row + 1, tq - 1);
                board[row][col] = false;
            }
        }

    }
```

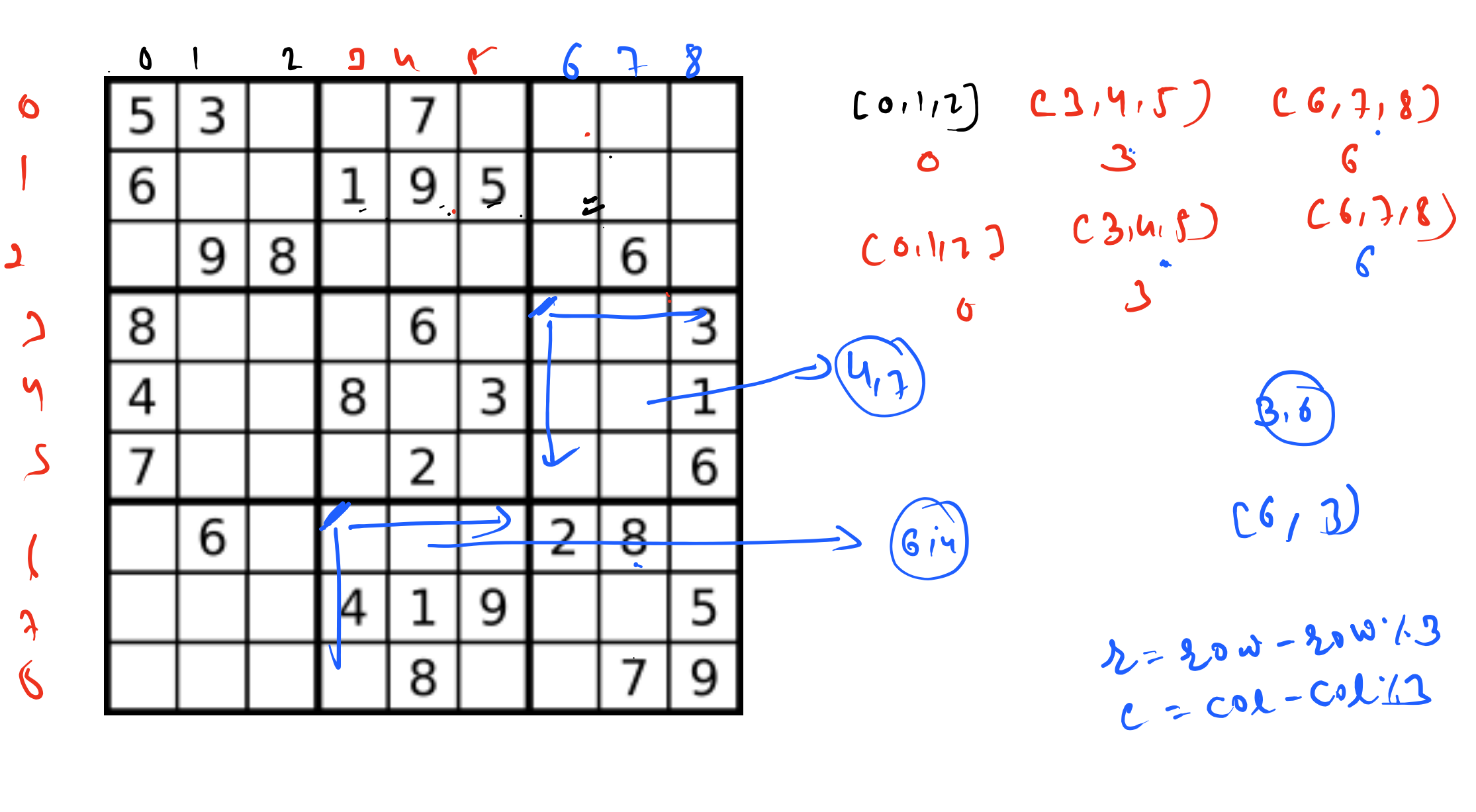

0 1 2 3
0,4
1,3
2,2
3,1
4,0
q

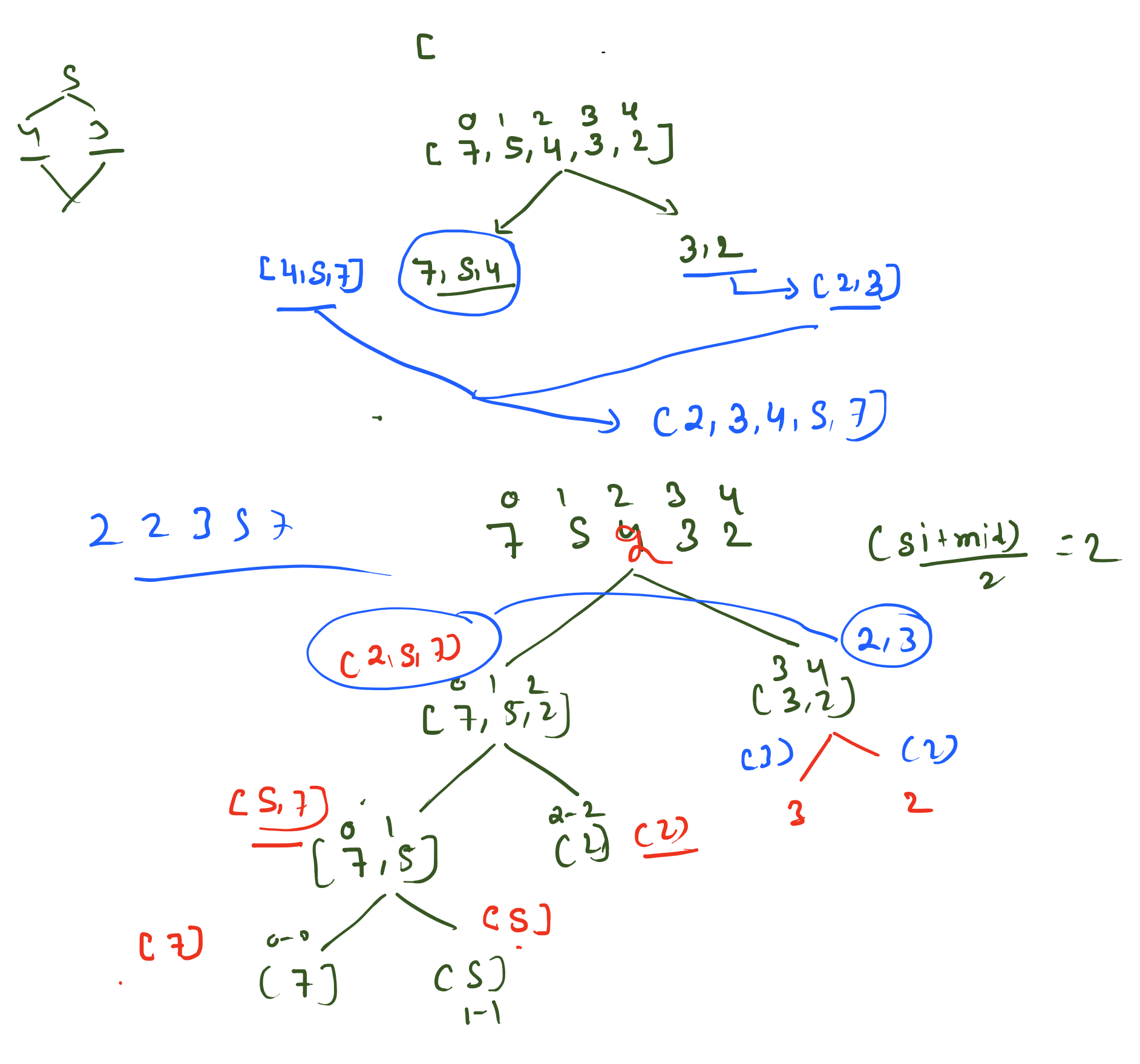



0 1 2 3 4 5 6 7 8
5 3 1 2 7 6 4 9 8
6 4 2 1 9 5
9 8 6
8 6 3
4 8 3 1
7 2 6
6 2 8
4 1 9 5
8 7 9
0,9
8,9
row, col+1
col==9
row++, col=0

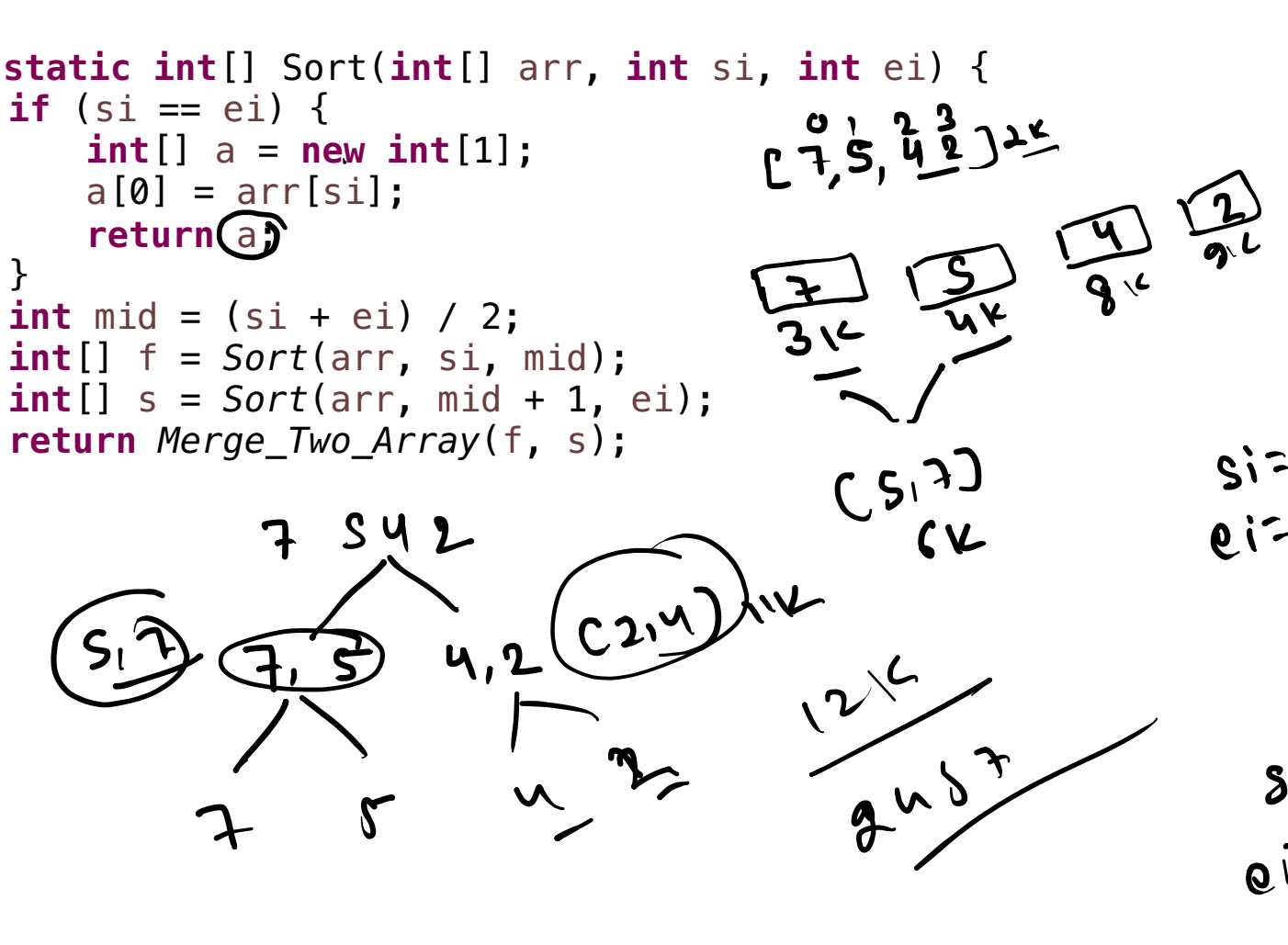
```java
public static boolean Print(int[][] grid, int row, int col) {
        if (col == 9) {
            row++;
            col = 0;
        }
        if(row==9) {
            return true;
        }
        if (grid[row][col] != 0) {
            return Print(grid, row, col + 1);
        } else {
            for (int val = 1; val <= 9; val++) {
                if (isitsafe(grid, row, col, val)) {
                    grid[row][col] = val;
                    boolean solve = Print(grid, row, col + 1);
                    if(solve) {
                        return solve;
                    }
                    grid[row][col] = 0;
                }
            }
        }
        return false;
    }
```

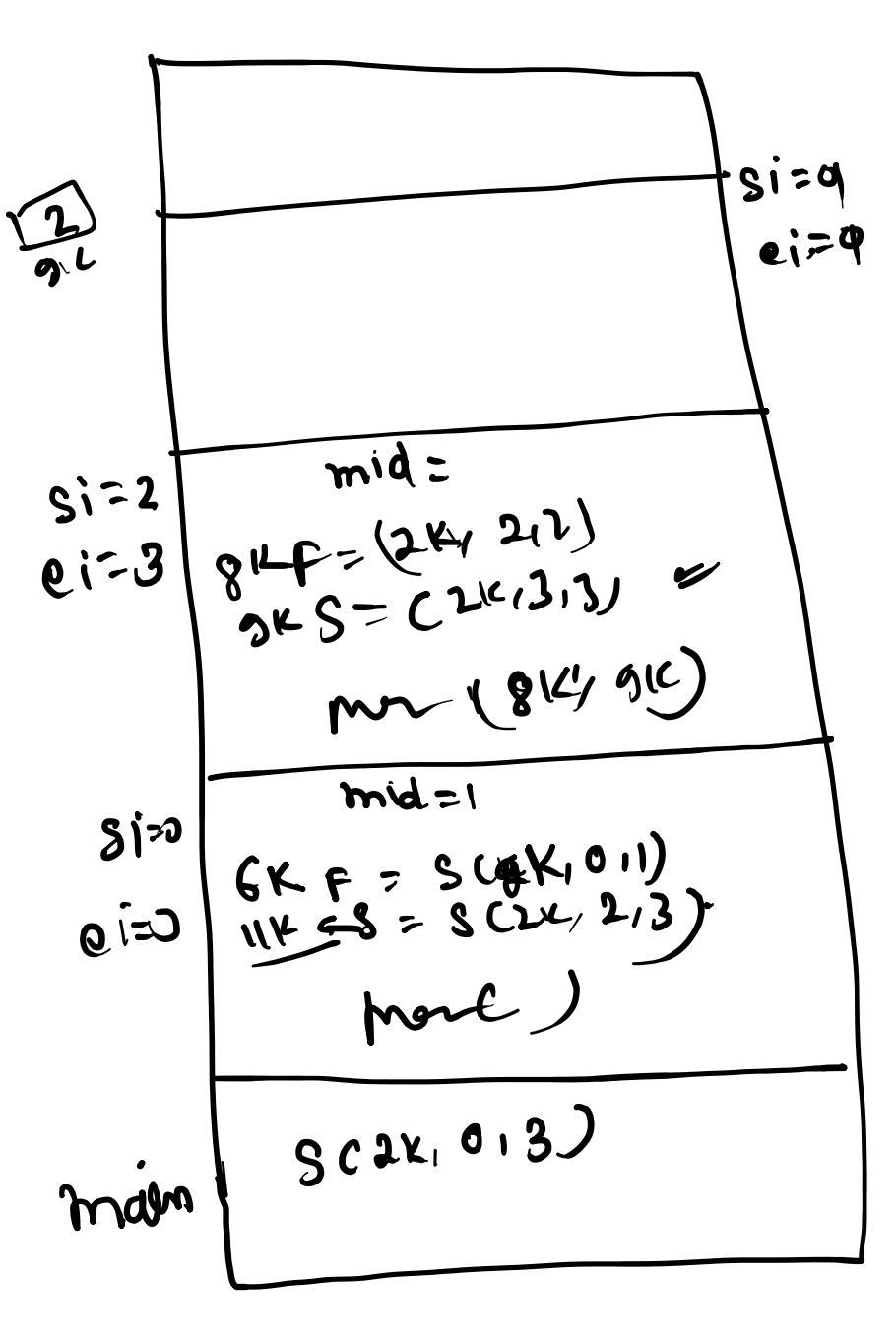

0 1 2 3 4 5 6 7 8
0,9
1,0
9

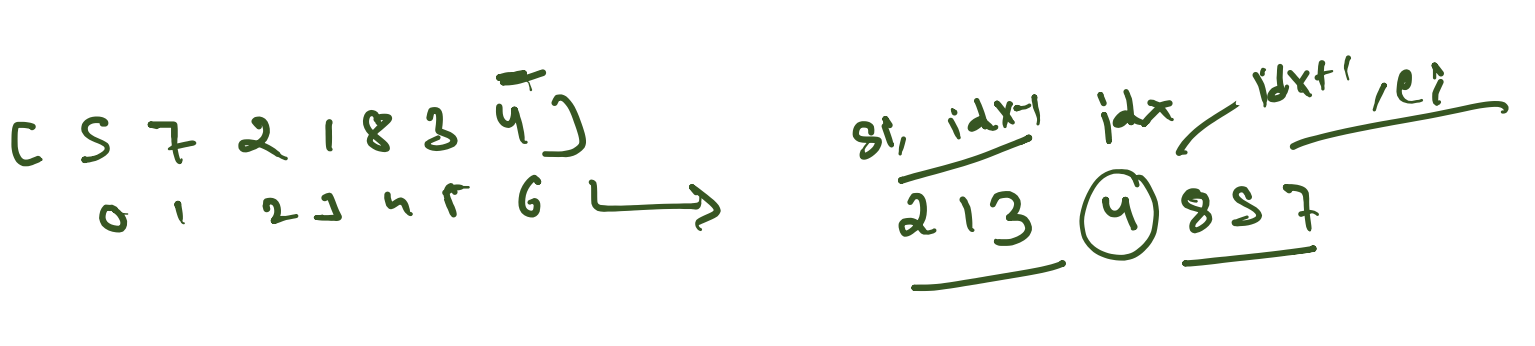

0 1 2 3 4 5 6 7 8
[0,1,2] [3,4,5] [6,7,8]
0 3 6
[0,1,2] [3,4,5] [6,7,8]
0 3 6
(4,7)
(6,4)
(3,6)
[6, 3]
r = row - row%3
c = col - col%3


5
4 3
0 1 2 3 4
[7,5,4,3,2]
[4,5,7]
[7,5,4]
3,2
[2,3]
[2,3,4,5,7]
2 2 3 5 7
(si+mid)/2 = 2
[2,5,7]
[7,5,2]
(2,3)
[3,2]
[3] [2]
3 2
[5,7]
[7,5]
[2] [2]
[7] [7]
[5] [5]

```java
public static int[] Sort(int[] arr, int si, int ei) {
        if (si == ei) {
            int[] a = new int[1];
            a[0] = arr[si];
            return a;
        }
        int mid = (si + ei) / 2;
        int[] f = Sort(arr, si, mid);
        int[] s = Sort(arr, mid + 1, ei);
        return Merge_Two_Array(f, s);

    }
```


[7,5,4,2] 2K
7 3K
5 4K
4 8K
2 9K
[5,7] 6K
7 5 4 2
5,7
7,5
4,2
[2,4] 11K
7 5 4 2
12K
2 4 5 7


si=9
ei=9
si=2
ei=3
mid=
8K f=(2K, 2,2)
9K S=(2K,3,3)
mr(8K, 9K)
si=0
ei=3
mid=1
6K f = S(2K,0,1)
11K S = S(2K, 2,3)
merge)
main
S(2K, 0,3)

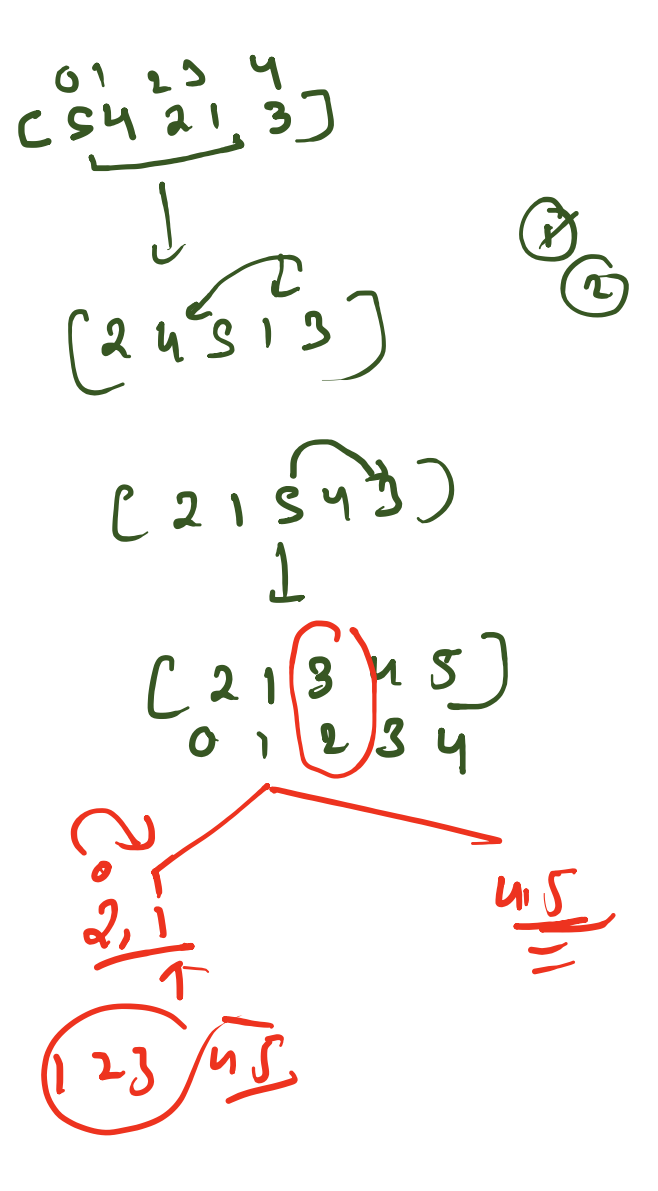

[5 7 2 1 8 3 4]
0 1 2 3 4 5 6
si, idx-1
idx
idx+1, ei
2 1 3 4 8 5 7

```java
public static void Sort(int[] arr, int si, int ei) {
        if (si >= ei) {
            return;
        }
        int idx = Partition(arr, si, ei);
        Sort(arr, si, idx - 1);
        Sort(arr, idx + 1, ei);

    }

    public static int Partition(int[] arr, int si, int ei) {
        int item = arr[ei];
        int idx = si;
        for (int i = si; i < ei; i++) {
            if (arr[i] < item) {
                int temp = arr[i];
                arr[i] = arr[idx];
                arr[idx] = temp;
                idx++;
            }
        }
        int temp = arr[ei];
        arr[ei] = arr[idx];
        arr[idx] = temp;
        return idx;

    }
```

→ ②

3

0


0 1 2 3 4
[5 4 2 1 3]
[2 4 5 1 3]
[2 1 5 4 3]
[2 1 3 4 5]
0 1 2 3 4
①
②
2,1
4,5
1 2 3 4 5